

APEX UPI – High-Concurrency PSP Switch & Ledger Engine

1. Introduction

Purpose

This High-Level Design (HLD) document describes the architecture, major components, deployment topology, service interactions, communication patterns, and infrastructure design of the APEX UPI platform.

The document translates the business requirements and software requirements into a technical architecture that can be implemented, deployed, operated, and maintained.

2. System Overview

APEX UPI is a distributed payment processing platform that simulates the architecture of a real UPI Payment Service Provider (PSP).

The system enables:

- UPI payment processing
- Account debit and credit
- Balance inquiry
- Transaction tracking
- Reconciliation
- Recovery workflows
- Audit logging
- Performance testing

The architecture is designed to provide:

- Scalability
- High Availability
- Fault Tolerance
- Event Traceability
- Operational Visibility

3. Architecture Principles

AP-01 Microservices

Business capabilities are implemented as independent services.

Benefits:

- Independent deployment
- Easier maintenance
- Better scalability

AP-02 Event-Driven Architecture

Communication occurs using Kafka events.

Benefits:

- Loose coupling
- Asynchronous processing
- High throughput

AP-03 Fault Tolerance

System supports:

- Retries
- Recovery workflows
- Reconciliation

AP-04 Auditability

Every transaction is traceable through:

- Events
- Ledgers
- Audit logs

AP-05 Idempotency

Duplicate transaction processing is prevented using:

- Transaction IDs
- RRN validation
- Event deduplication

4. Deployment Architecture

The platform is deployed across five virtual machines.

VM-1 : PSP Switch

Purpose

Acts as the entry point and orchestration engine.

Components

- PSP API Gateway
- Transaction Orchestrator
- Validation Layer
- PSP Ledger Manager
- Kafka Producer/Consumer

Responsibilities

- Accept payment requests
- Validate transactions
- Generate Transaction ID
- Generate RRN
- Publish events
- Track transaction lifecycle

VM-2 : Banking Switch

Purpose

Acts as middleware between PSP and CBS.

Components

- Banking Router
- Banking Validation Service
- CBS Request Coordinator

Responsibilities

- Route banking requests
- Validate banking rules
- Manage retries
- Forward requests to CBS

VM-3 : Core Banking System (CBS)

Purpose

Maintains customer financial data.

Components

- Account Service
- Debit Service
- Credit Service
- Balance Inquiry Service
- Reversal Service

Responsibilities

- Maintain balances
- Process debits
- Process credits
- Handle reversals

VM-4 : NPCI Adapter

Purpose

Simulates NPCI network processing.

Components

- NPCI Router
- NPCI Validation Layer
- Settlement Simulator

Responsibilities

- Validate payee routing
- Simulate NPCI behavior
- Route requests
- Return responses

VM-5 : Load Runner

Purpose

Performance and stress testing.

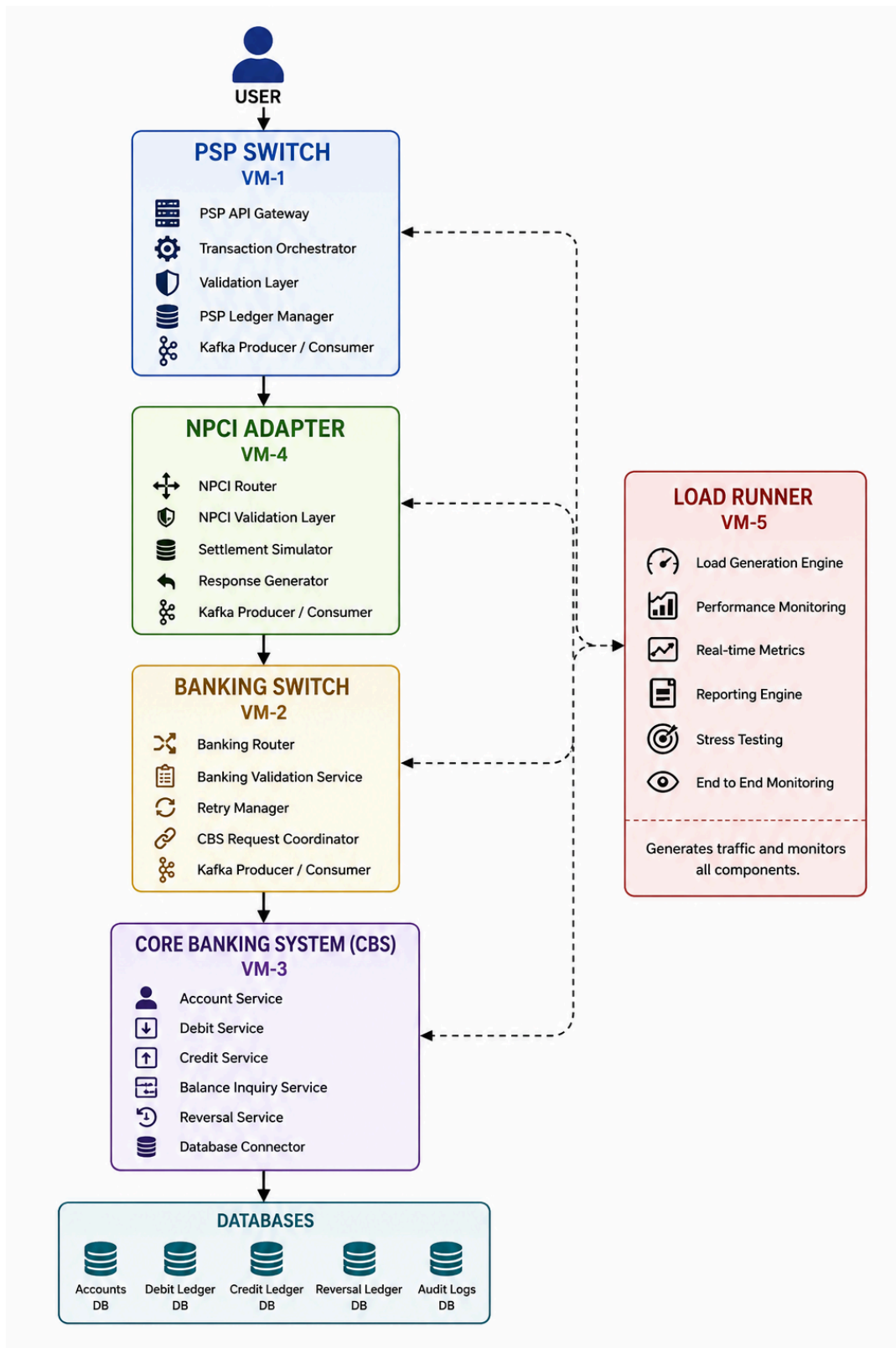
Components

- Load Generation Engine
- Monitoring Tools
- Reporting Engine

Responsibilities

- TPS testing
- Stress testing
- Latency measurement
- Throughput measurement

5. High-Level Architecture Diagram



6. Service Architecture

PSP Switch

Business Purpose

Acts as the transaction orchestration layer.

Major Functions

- Payment initiation
- Validation
- Status tracking
- Event publishing

Inputs

- Payment Request
- Balance Request
- Status Request

Outputs

- Transaction Status
- Payment Confirmation

Banking Switch

Business Purpose

Coordinates banking operations.

Major Functions

- Debit request routing
- Credit request routing
- CBS integration

Core Banking System

Business Purpose

Source of truth for customer balances.

Major Functions

- Debit processing
- Credit processing
- Balance inquiry
- Reversal execution

NPCI Adapter

Business Purpose

Simulates NPCI transaction network.

Major Functions

- Routing validation
- Settlement simulation
- Response generation

7. Communication Architecture

The system uses two communication mechanisms.

Synchronous Communication

Protocol:

REST API

Used for:

- Payment initiation
- Balance inquiry
- Transaction status

Asynchronous Communication

Protocol:

Apache Kafka

Used for:

- Payment events
- Ledger updates
- Recovery workflows
- Audit events

8. Kafka Architecture

Purpose

Provides asynchronous communication between services.

Topics

Payment Events

upi.transactions.initiated

Debit Events

upi.cbs.debit

Credit Events

upi.cbs.credit

Reversal Events

upi.cbs.reversal

Status Events

upi.transactions.status

Dashboard Events

Upi.dashboard.events

9. Producer Consumer Matrix

Topic	Producer	Consumer
upi.transactions.initiated	PSP Switch	NPCI Adapter
upi.cbs.debit	NPCI Adapter	Banking Switch
upi.cbs.credit	Banking Switch	CBS
upi.transactions.status	CBS	PSP Switch
upi.dashboard.events	All Services	Dashboard

10. Database Architecture

Accounts Table

Stores customer account details.

Attributes

VPA
Account Number
Balance
Bank
Status

Debit Ledger

Stores debit transactions.

Attributes

Txn ID
RRN
Amount
Balance After
Status
Timestamp

Credit Ledger

Stores credit transactions.

Attributes

Txn ID

Amount

Status

Timestamp

Reversal Ledger

Stores rollback operations.

Attributes

Txn ID

Reason

Status

Timestamp

11. Payment Processing Flow

Step 1

User initiates payment.

Step 2

PSP Switch validates:

- VPA
- Amount
- Format

Step 3

Transaction ID generated.

Example:

TXN202600001

Step 4

RRN generated.

Example:

RRN123456789

Step 5

Transaction event published.

Step 6

NPCI Adapter receives event.

Step 7

Routing validated.

Step 8

Banking Switch receives request.

Step 9

CBS performs debit.

Step 10

CBS performs credit.

Step 11

Ledgers updated.

Step 12

Success event published.

Step 13

PSP Switch returns success.

12. Balance Inquiry Flow

Customer
↓
PSP Switch
↓
Banking Switch
↓
CBS
↓
Balance Response

13. Reversal Flow

Triggered when:

- Credit failure
- Timeout
- Service crash

Flow:

Debit Success
↓
Credit Failure
↓
Reversal Initiated
↓

CBS Reversal

↓

Ledger Update

↓

Reversal Complete

14. Recovery Architecture

Recovery engine handles:

NPCI Timeout

Action:

Retry

CBS Failure

Action:

Reversal

Kafka Failure

Action:

Retry

DLQ

15. Reconciliation Architecture

Purpose:

Identify inconsistencies between:

- PSP Ledger
- Banking Ledger
- CBS Records

Checks:

- Missing debit
- Missing credit
- Duplicate transaction
- Partial transaction

Actions:

- Recovery
- Reversal
- Manual review

16. Security Architecture

Authentication

Service-to-service authentication.

Authorization

Role-based access.

Data Integrity

Transaction IDs validated.

Audit Logging

Every action logged.

17. Monitoring Architecture

Metrics captured:

Application Metrics

- TPS
- Success Rate
- Failure Rate

Infrastructure Metrics

- CPU
- Memory
- Disk

Kafka Metrics

- Consumer Lag
- Topic Throughput

Database Metrics

- Query Latency
- Connections

18. Performance Requirements

Metric	Target
Payment Response	< 5 sec
Balance Inquiry	< 2 sec
Availability	99.9%
Data Loss	0
Transaction Accuracy	100%

19. Failure Handling Strategy

Failure	Handling
Invalid VPA	Reject
Insufficient Balance	Reject
NPCI Timeout	Retry
CBS Timeout	Reversal
Kafka Failure	Retry + DLQ

20. Assumptions

- Kafka available
- PostgreSQL available
- Network stable
- NPCI simulated
- CBS simulated
- Unique transaction IDs
- Reliable event delivery

21. Future Enhancements

- Multi-bank support
- Real NPCI integration
- Active-active deployment
- Kubernetes deployment
- AI fraud detection
- Real-time analytics
- Multi-region failover

22. Conclusion

The APEX UPI High-Level Design provides a scalable, fault-tolerant, event-driven architecture capable of processing UPI transactions across five distributed virtual machines. The design emphasizes reliability, ledger consistency, recovery mechanisms, reconciliation workflows, and operational observability, closely mirroring enterprise-grade payment processing systems used in modern financial institutions.